

A quick intro to python (and numpy)

Prof. Alexander Binder

October 14, 2025

These slides are a mix of simple topics (e.g. lists,dictionaries, for-loops) and advanced topics (call by object reference, when passing any object to a function)

- ① Matrix-vector and Matrix-Matrix multiplication
- ② Python Basics with some notable sources of mistakes
- ③ Strings and Bytes
- ④ Dateien ohne high level funktionen
- ⑤ numpy ...
- ⑥ Basic 2d plotting in matplotlib

Given a matrix A of shape (n, d) and a matrix B of shape (d, f) , their multiplication AB is defined as a matrix of shape (n, f) and its component $(AB)_{i,k}$ at row i and column k is given as

$$(AB)_{i,k} = A_{(i,:)} \cdot B_{(:,k)} = \sum_{r=0}^{d-1} A_{i,r} B_{rk}$$

as an inner product between the i -th row of the left matrix and the k -th column of the right matrix.

Important: the number or dimensions in the second axis of A must be equal to the number or dimensions in the first axis of B . Otherwise matrix multiplication is not possible.

Therefore a matrix- matrix multiplication is a matrix consisting of inner products:

$$AB = \begin{bmatrix} A_{(0,:)} \cdot B_{(:,0)} & A_{(0,:)} \cdot B_{(:,1)} & A_{(0,:)} \cdot B_{(:,2)} & \dots & A_{(0,:)} \cdot B_{(:,f-1)} \\ A_{(1,:)} \cdot B_{(:,0)} & A_{(1,:)} \cdot B_{(:,1)} & A_{(1,:)} \cdot B_{(:,2)} & \dots & A_{(1,:)} \cdot B_{(:,f-1)} \\ A_{(2,:)} \cdot B_{(:,0)} & A_{(2,:)} \cdot B_{(:,1)} & A_{(2,:)} \cdot B_{(:,2)} & \dots & A_{(2,:)} \cdot B_{(:,f-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A_{(n-1,:)} \cdot B_{(:,0)} & A_{(n-1,:)} \cdot B_{(:,1)} & A_{(n-1,:)} \cdot B_{(:,2)} & \dots & A_{(n-1,:)} \cdot B_{(:,f-1)} \end{bmatrix}$$

Memorizing

- ⦿ Matrix-Matrix multiplication results in a matrix, if shapes are permissible:
 $(n, d)(d, f) \rightarrow (n, f)$.
- ⦿ The component $(AB)_{i,k}$ at row i and column k is given as the inner product between row i of the left matrix and column k of the right matrix.
- ⦿ That also tells you which axes must match in dimensionality:
Left matrix - the number of columns = the dimensionality of the second axis.
Right matrix - the number of rows = the dimensionality of the first axis.

Given a matrix of shape (n, d) and a vector of dimensionality d , the multiplication Ax of them (with x being on the right hand side) is defined as a vector of length n such that

$$\begin{aligned} Ax &= \begin{bmatrix} a_{0,0} & \dots & a_{0,d-1} \\ a_{1,0} & \dots & a_{1,d-1} \\ \vdots & \ddots & \vdots \\ a_{n-1,0} & \dots & a_{n-1,d-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \dots \\ x_{d-1} \end{bmatrix} \\ &= \begin{bmatrix} \begin{bmatrix} a_{0,0} & \dots & a_{0,d-1} \end{bmatrix} \cdot x \\ \begin{bmatrix} a_{1,0} & \dots & a_{1,d-1} \end{bmatrix} \cdot x \\ \vdots \\ \begin{bmatrix} a_{n-1,0} & \dots & a_{n-1,d-1} \end{bmatrix} \cdot x \end{bmatrix} \end{aligned}$$

Thus, Ax is a vector and the k -th component of vector Ax is given as an inner product

$$\begin{aligned}(Ax)_k &= A[k, :] \cdot x \\ &= (a_{k,0} \dots a_{k,d-1}) \cdot x \\ &= \sum_{r=0}^{d-1} a_{k,r} x_r\end{aligned}$$

between the k -th row of A and vector x . The matrix multiplication with a vector from the right is only defined, if the number of columns in A is equal to the dimensionality of the vector x .

Given a matrix of shape (n, d) and a vector of dimensionality n , the multiplication $x^\top A$ of them (with $x^\top$ being on the left hand side) is defined as a vector of length d such that

$$\begin{aligned}
 x^\top A &= \begin{bmatrix} x_0, x_1, \dots, x_{n-1} \end{bmatrix} \begin{bmatrix} a_{0,0} & \dots & a_{0,d-1} \\ a_{1,0} & \dots & a_{1,d-1} \\ \vdots & \ddots & \vdots \\ a_{n-1,0} & \dots & a_{n-1,d-1} \end{bmatrix} \\
 &= \begin{bmatrix} x \cdot \begin{bmatrix} a_{0,0} \\ \vdots \\ a_{n-1,0} \end{bmatrix} & x \cdot \begin{bmatrix} a_{0,1} \\ \vdots \\ a_{n-1,1} \end{bmatrix} & \dots & x \cdot \begin{bmatrix} a_{0,d-1} \\ \vdots \\ a_{n-1,d-1} \end{bmatrix} \end{bmatrix}
 \end{aligned}$$

Thus, $x^\top A$ is a vector and the k -th component of vector $x^\top A$ is given as an inner product

$$\begin{aligned}(x^\top A)_k &= x \cdot A[:, k] \\ &= (a_{0,k} \dots a_{n-1,k}) \cdot x \\ &= \sum_{r=0}^{n-1} x_r a_{r,k}\end{aligned}$$

Consequence:

inner products as matrix vector multiplication

The inner product $x \cdot y$ of two column-shaped vectors can be written in matrix-vector multiplication notation as

$$x \cdot y = \begin{bmatrix} x_0 & x_1 & \dots & x_{d-1} \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{d-1} \end{bmatrix} = x^\top y$$

where $x^\top$ is the transpose of a vector or matrix x .

The transpose was used here to convert the column-shaped vector into a row-shaped vector.

- 1 Matrix-vector and Matrix-Matrix multiplication
- 2 Python Basics with some notable sources of mistakes
- 3 Strings and Bytes
- 4 Dateien ohne high level funktionen
- 5 numpy ...
- 6 Basic 2d plotting in matplotlib

Section on: **Python Basics with some notable sources of mistakes**

- ⦿ interpreted language

```
#hello.py
```

```
a=5
```

```
print(a)
```

```
print('hello, un-world\n')
```

```
#end of hello.py
```

```
binder@binder-ThinkPad-X390:~$ python3 hello.py
```

- use __main__ as entry point to see where the code starts:
- can use functions
- indentation matters: to mark a block, here, to mark the end of a function

#hello2.py

```
def run():  
    a=5  
  
    print(a)  
    print('hello, un-world\n')  
  
if __name__=='__main__':  
    run()
```

- string, content immutable, concat ok (+)

```
def run():  
    a = 'xyz'  
    b = 'a1 2'  
    a+=b  
    print(a)  
    #a[0]='j'  
if __name__=='__main__':  
    run()
```

- list [] or *list()*, like a vector, but for mixed types
- content mutable, concat ok (+)

```
def run():  
    a = []  
    a.append(5)  
    a.append('C')  
    print(a)  
    b = [2,3]  
    a += b  
    print(a)  
    a[1] = 'x' #content is mutable, works!  
if __name__ == '__main__':  
    run()
```

- ⦿ tuple () or *tuple()*
- ⦿ content immutable, concat ok (+)

```
def run():  
    a = (3,5,'c')  
    b = (2,1)  
    a +=b  
    print(a)  
    # a[0]= 5  
if __name__=='__main__':  
    run()
```

- tuple and strings are often called immutable in the internet, but they can be modified using concatenation
- immutable refers to the already existing fields!

- dictionary - key-value storage
- content mutable, useful for options/settings

```
def run():
    dic = {}
    d2 = dict()
    d2['a']=3
    v1=d2.get('b')
    v2=d2.get('a')

    dic[3.8]=7.2
    x=3.8
    if x in dic:
        print('has key', x)
    else:
        print('has no key', x)

if __name__=='__main__':
    run()
```

- ⦿ `type(object)`
- ⦿ `isinstance( object, class)` - returns true also for derived classes (!)

```
def run():  
    a = (3,5,'c')  
    print(type(a))  
    print(type(a[0]))  
    print(type(a[2]))  
  
if __name__=='__main__':  
    run()
```

- ⦿ structure: keyword def

```
def functionname(arg1,arg2,...,argn):  
    #function body  
    return value1,return value2,...,valuen
```

- ⦿ functions can take multiple arguments, return multiple values

#hello3.py

```
def run(a):  
    a+=3  
    b= a*2  
    c= b//3  
    return b,c  
  
if __name__=='__main__':  
    a=2  
    b,c = run(a)  
    print(a,b,c)
```

- ⦿ Answer: **Neither!!**
- ⦿ Python uses "call by object reference"
- ⦿ What ?

<https://www.youtube.com/watch?v=NYvwLLxKqhM>

- ⦿ see `callbyobjectreference.py` and `callbyobjectreference2.py` to understand it

- see `callbyobjectreference.py` and `callbyobjectreference2.py` to understand it
- you **cannot change an object passed as a whole** to a function : the change is made only in the copy of the object within the function, but does not change the object outside the function call
- you **can change mutable members of an object**
 - modifying list elements, append to a list - works
 - member structures of an object passed to a function - works
 - contents of dictionaries, sets - works
- tuples and **strings** cannot be permanently changed outside of the function call when using concat (`str1+ = str2`, `tuple1+ = tuple2`) within the function call. Why not ? Concatenation would overwrite the whole object by a copy of it! Any such change would be only valid in the copy within the function (due to the rule “cannot mod an object as a whole”).
 - but there is a way to make the changed copy visible outside ... via the return statement

- see `callbyobjectreference2_extra.py`, `callbyobjectreference3.py` to understand it
- we said before: when you change an object as a whole, then the change is made only in the internal copy inside the function.
You can **return** the changed internal copy using the return statement. This will make a changed object available outside of the function.
- tuples and strings: when passed to a function, **the internal copy of it** can be modified using concatenation (with the same type) for both types internally, and then returned by the function !
The returned copy will have the the change made in it, whereas the argument to the function will be unchanged

⊙ ...

#hello3.py

```
if __name__=='__main__':
```

```
    for i in range(7):
```

```
        print('i',i)
```

```
    print('-----')
```

```
    somelist = [1,'a',2]
```

```
    somelist.append(3)
```

```
    for el in somelist:
```

```
        print(el)
```

```
    print('-----')
```

```
    somelist.extend(['xyz',999])
```

```
    for el in somelist:
```

```
        print(el)
```

- ⦿ This does not work. Same reason as for function arguments!

```
if __name__=='__main__':
```

```
    somelist = [1,'a',2]
```

```
    for el in somelist:
```

```
        el = 'u'
```

```
    print(somelist)
```

- ⦿ This does work. Same reason as for function arguments!

```
class bla():
    def __init__(self):
        self.value=3

if __name__=='__main__':

    somelist = [bla(),bla(),bla()]
    for i,el in enumerate(somelist):
        el.value = 9 + 3*(i+1)

    for i,el in enumerate(somelist):
        print( "i-th object",i , 'value:',el.value)
```

What is it about ?

- ⦿ to NOT code like this: <https://www.youtube.com/shorts/hfrU0ENHQ3U>
- ⦿ specifies input and output types in functions

- for static type checkers like mypy, supports builtin types such as list, dict, functions
- see hello6.py

```
def seed(a: float) -> int: #expects a float, returns an int
    return int(3)
class bla():
    pass
def returnsbla() -> bla:
    b = bla()
    return b
```

- cheat sheet:
https://mypy.readthedocs.io/en/stable/cheat_sheet_py3.html

- ⦿ `def __init__(self, var1, var2):` - class constructor
- ⦿ use `self.var` to access a variable of the class instance within an instance method

```
class bla():  
    def __init__(self, x):  
        self.value = x  
    def addtovalue(self, x):  
        self.value += x  
        return self.value
```

```
x=3  
b=bla(x)  
b.value += 1.0  
print(b.value)  
b.addtovalue(2.0)  
print(b.value)
```

Python is weakly typed. A variable can hold anything

```
class bla():  
    def __init__(self):  
        self.value= 3.8  
  
def f(a,b):  
    return a+ b**2  
  
b=bla()  
b.value = f #works, but is Sh*t code  
z= b.value(-1.0,3.0)  
print(z)  
b.value = bla()  
print(b.value.value)
```

- ① Matrix-vector and Matrix-Matrix multiplication
- ② Python Basics with some notable sources of mistakes
- ③ Strings and Bytes
- ④ Dateien ohne high level funktionen
- ⑤ numpy ...
- ⑥ Basic 2d plotting in matplotlib

Bytes:

- sequence of 1 byte elements, each element is in $[0, 255]$
- not capable to represent unicode
- length of string via `len()` equal to number of visible elements
- for encoding of data, for payload like network packets

```
msg2= b'this is a bytes-type' #the b before the '' defines a py3 bytes type
print('\n\n')

print('len(msg2)', len(msg2))
```

String:

- sequence of ASCII OR unicode elements, each el can be larger than 1 byte
- length via `len()` NOT equal to number of visible elements
- for looping does not print the same as printing the whole string (cf. C/C++)
- byte string from unicode string may have varying length per visible symbol !
- for pretty printing in many languages

```
msg2= 'this is a str-type' #without the b before the '' defines a py3 string type  
print('\n\n')
```

```
print('len(msg2)', len(msg2))
```

see `stringvsbytes2.py` and `unicodeambiguity.py` for a length mismatch!

- ⦿ bytes decode(...) into string
- ⦿ string encode() into bytes

```
bytesobj=b'\xe6\x00\xb4'  
stringobj=bytesobj.decode('utf-8')  
print(stringobj)  
newbytesobj = stringobj.encode()  
print(bytesobj, '|vs|',newbytesobj)
```

see stringvsbytes2.py and unicodeambiguity.py for a length mismatch!

- ① Matrix-vector and Matrix-Matrix multiplication
- ② Python Basics with some notable sources of mistakes
- ③ Strings and Bytes
- ④ Dateien ohne high level funktionen
- ⑤ numpy ...
- ⑥ Basic 2d plotting in matplotlib

- ⊙ with Umgebung - nutzt methoden `__enter__` (um sicherzustellen, dass das Öffnen der Datei funktioniert) und `__exit__` (um den Dateihandle zu schliessen)
- ⊙ 'r' - read text mode

```
with open('datei.txt','r') as f:  
    str = f.readline() # eine Zeile  
    str2 = f.readline() # eine weitere Zeile
```

- ⦿ with Umgebung - nutzt methoden `__enter__` (hier: um sicherzustellen, dass das Öffnen der Datei erfolgreich ist) und `__exit__` (hier: um den Dateihandle zu schliessen)
- ⦿ `'rb'` - read binary mode
- ⦿ see below for pickling of multiple things !!

```
import struct
with open('datei.txt','rb') as f:
    var= struct.unpack('i', f.read(4))
# reads a 4 byte into a bytes string, then converts the bytes object into a 4 byte integer
```

- ⊙ 'w' - write text mode

'{:5.2f}'.format(var) + '\n' wandelt die Gleitkommavariablen in einen string um, mit max 2 nachkommastellen und haengt ein newline an '\n'

```
var = 14.22645567567
with open('datei.txt','w') as f:
    f.write( '{:5.2f}'.format(var) + '\n') #
with open('datei.txt','r') as f:
    stringobj = f.readline()
    stringobj = stringobj.rstrip() # remove the newline !!
    floatvar = float(stringobj)
```

- ⊙ text writing and reading multiple into a line

```
var = 14.22645567567
var2= 3.5
stro = 'xyz'
with open('datei.txt','w') as f:
    f.write( '{:5.2f} {:5.2f} {}'.format(var,var2,stro) +'\n')
with open('datei.txt','r') as f:
    stringobj = f.readline()
    stringobj = stringobj.rstrip() # remove the newline !!
    listofstr = stringobj.split()
    floatvar = float(listofstr[0])
    floatvar2 = float(listofstr[1])
    stro2 = listofstr[2]
```

- ⦿ for multiple binary values: use python's builtin pickle !!
- ⦿ Classes, functions, and methods cannot be pickled <https://wiki.python.org/moin/UsingPickle>
- ⦿ you must check that dictionaries contain only pickleable objects!

```
import pickle
var = 14.22645567567
var2= 3.5
stro = 'xyz'
with open('datei.txt','wb') as f:
    pickle.dump([var , var2, stro], f)

with open('datei.txt','rb') as f:
    vals = pickle.load(f)
print(vals)
```

Malicious Pickle files!

fuer ein Beispiel:

<https://davidhamann.de/2020/04/05/exploiting-python-pickle/>

- ◉ `hasattr(...)`, `getattr(...)` - praktisch um herauszufinden, ob ein Objektinstanz eine Variable oder Methode besitzt
- ◉ `try - except - else` Programmierung <https://stackoverflow.com/questions/855759/what-is-the-intended-use-of-the-optional-else-clause-of-the-try-statement-in>
 - finde heraus, ob etwas geht, mache weiter falls ja (uses cases: Typ conversionen, Dateien lesen, packages importieren, objekt attribute ...)

```
s = 'x3.5'
def tofloat(s: str) -> float:
    try:
        v= float(s)
    except:
        v= None
    return v

v= tofloat(s)
print(v)

try:
    from somepackage import somefct
except ImportError:
    do_workaround()
else:
    v = somefct(x,y,z)
```

- ⊙ `functools.partial(...)` um Funktionen mit Argumenten zu initialisieren

```
import functools
```

```
data = [ (-1,2), (2,5), (3,5)]
```

```
def distance(x,y):  
    return (x[0]-y[0])**2 + (x[1]-y[1])**2
```

```
refvec = (1.98,5.03)  
dlistvector= functools.partial(distance, refvec )
```

```
data.sort(key = dlistvector)  
print(data)
```

- ① Matrix-vector and Matrix-Matrix multiplication
- ② Python Basics with some notable sources of mistakes
- ③ Strings and Bytes
- ④ Dateien ohne high level funktionen
- ⑤ numpy ...
- ⑥ Basic 2d plotting in matplotlib

- ⊙ library for arrays and linear algebra

```
a= np.ones(( 7,5)) # a matrix of shape 7,5  
print(a.shape)
```

- ⊙ library for arrays and linear algebra

```
a= np.zeros(( 3,64,64)) # a 3d array of shape 3,64,64 (e.g. RGB image!)  
print(a.shape)
```

- ⦿ Why is that bad code ?

```
import numpy as np
rng = np.random.default_rng()
a= rng.integers(low=0,high=10,size=(3,5))
b= rng.integers(low=0,high=10,size=(3,5))
c1 = a+b # element-wise addition, shapes must match
```

SEED RNGs !!

Always SEED random generators. → so that experiments become reproducible science



[https://commons.wikimedia.org/wiki/File:](https://commons.wikimedia.org/wiki/File:An_alchemist_in_his_laboratory._Oil_painting_by_a_follower_o_Wellcome_V0017688.jpg)

[An_alchemist_in_his_laboratory._Oil_painting_by_a_follower_o_Wellcome_V0017688.jpg](https://commons.wikimedia.org/wiki/File:An_alchemist_in_his_laboratory._Oil_painting_by_a_follower_o_Wellcome_V0017688.jpg)

- ⊙ library for arrays and linear algebra

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.integers(low=0,high=10,size=(3,5))
b= rng.integers(low=0,high=10,size=(3,5))
c1 = a+b # element-wise addition, shapes must match
c2= a*b # element-wise product, shapes must match
```

- ⊙ inner product

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.integers(low=0,high=10,size=(3))
b= rng.integers(low=0,high=10,size=(3))
c  = np.inner(a,b) # inner product
```

- ⊙ inner product

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.integers(low=0,high=10,size=(3,5))
b= rng.integers(low=0,high=10,size=(3,5))
c = np.inner(a,b) # 3 inner products over the last dimension of length 5!
# c[0,0]=np.inner(a[0,:], b[0,:])
# c[2,2]=np.inner(a[2,:], b[2,:])
# c[i,k]=np.inner(a[i,:], b[k,:])
```

- ⊙ matrix multiplication

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.integers(low=0,high=10,size=(3,5))
b= rng.integers(low=0,high=10,size=(5,6))
# shapes must satisfy the rule for matmul
# (i,k) (k,n)
c = np.matmul(a,b) #
c2 = a @ b
print(c.shape)
```

- for more general combinations of multiplication and summation: einsum
- allows for n-arrays to define what is paired and multiplied (indices matching across arrays), and what is summed out (indices missing on the right hand side)

<https://rockt.github.io/2018/04/30/einsum>

- ⊙ statistics over arrays <https://numpy.org/doc/stable/reference/routines.statistics.html>

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.integers(low=0,high=10,size=(3,5,7))
b = np.mean(a, axis=(0,2))
c = np.var(a, axis=(1))
median = np.quantile(a, q=0.5, axis=(0,2))
```

- ⊙ random selection and shuffling <https://numpy.org/doc/stable/reference/random/generator.html>

```
import numpy as np
a=np.arange(5)
rng = np.random.default_rng(seed=3)
parts = rng.choice(a,size=3)
b=rng.permutation(a) # makes a permuted copy
print(b,a)
rng.shuffle(a) # in-place!!, #there is also permuted
print(a)
```

- ⊙ drawing from simple random distributions

<https://numpy.org/doc/stable/reference/random/generator.html>

```
import numpy as np
```

```
rng = np.random.default_rng(seed=3)
```

```
a= rng.random(size=(2,2)) # numbers in [0,1)
```

```
b= rng.integers(low=0, high=12, size=(3,3)) # integers 0 to 12-1
```

- ⊙ drawing from non-trivial random distributions
<https://numpy.org/doc/stable/reference/random/generator.html>

```
import numpy as np
```

```
rng = np.random.default_rng(seed=3)  
b = rng.beta(a=0.5, b=1.5, size=(2,2))
```

- ⊙ eigenvalues and eigenvectors of a square array
<https://numpy.org/doc/stable/reference/routines.linalg.html>

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.random(size=(5,5))
b = a.T @ a #makes b symmetric, so all eigenvalues real-valued
w,v = np.linalg.eig(b)
```

- ⊙ solve $Ax = b$ given a matrix $A \in \mathbb{R}^{d \times d}$ and a bias vector b
- ⊙ get rank <https://numpy.org/doc/stable/reference/routines.linalg.html>

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.random(size=(5,5))
A = np.matmul(a.T,a) #makes b non-negative definite
b = rng.integers(low=0,high=10,size=(5,1))
x= np.linalg.solve(A,b)
r = np.linalg.matrix_rank(A)
```

- ⊙ sort, argsort (e.g. find the top-5 predicted classes in a prediction)
<https://numpy.org/doc/stable/reference/routines.sort.html>

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.integers(low=0,high=10,size=(5,3))
inds = np.argsort(a,axis=0)
```

- ⊙ Indexing syntax: [start:stop:step]
- ⊙ stop is NOT included

```
import numpy as np
```

```
a = np.arange(50)
```

```
b= a[0:3]
```

```
print(b)
```

```
b= a[0:5]
```

```
print(b)
```

```
b= a[2:6]
```

```
print(b)
```

```
b= a[0:6:2] #step 2, 6 cannot be included!
```

```
print(b)
```

- ◉ Indexing syntax: [start:stop:step]
- ◉ stop is NOT included
- ◉ negative at start and stop means: count from the end, start at last n , stop at last k

```
import numpy as np
```

```
a = np.arange(50)
```

```
b= a[2:6]
```

```
print(b)
```

```
b= a[2:-1] # stop at last 1 (= drop the last 1)
```

```
print(b)
```

```
b= a[2:-5] # stop at last 5 (= drop the last 5)
```

```
print(b)
```

```
b= a[-6:] # start at last 6
```

```
print(b)
```

```
b= a[-6:-5] # start at last 6, stop at last 5
```

```
print(b)
```

- ⦿ Indexing syntax: [start:stop:step]
- ⦿ stop is NOT included
- ⦿ negative at step means: reverse order

```
import numpy as np
```

```
a = np.arange(50)
```

```
b= a[2:6:2] #step 2, 6 cannot be included!  
print(b)
```

```
b= a[:,2] #using defaults: 0,end, and step=2  
print(b)
```

```
b= a[5::3] #start at element 5, using default for the end, and step=3  
print(b)
```

```
b= a[::-1] #reverse order  
print(b)
```

```
b= a[::-2] #reverse order, step =2  
print(b)
```

- ⊙ np.save and np.load

```
import numpy as np
rng = np.random.default_rng(seed=3)
a= rng.integers(low=0,high=10,size=(5,3))
np.save('bla.np',a)
```

- ⦿ <https://www.datacamp.com/cheat-sheet/numpy-cheat-sheet-data-analysis-in-python>
- ⦿ broadcasting ?

- ① Matrix-vector and Matrix-Matrix multiplication
- ② Python Basics with some notable sources of mistakes
- ③ Strings and Bytes
- ④ Dateien ohne high level funktionen
- ⑤ numpy ...
- ⑥ Basic 2d plotting in matplotlib

Hier gibt es Beispielcode! Einfach modifizieren

<https://matplotlib.org/stable/gallery/index.html>

see `plot_somelddata_v2.py`

```
fig = plt.figure() # creates a figure which can contain subfigures
fig.suptitle('Main figure title')
fig.suptitle('Main figure title', fontsize=22)

ax1 = fig.add_subplot(311) # add subfigure
ax1.set_title('Subplot 1 title') # set subfigure title
ax1.set_title('Subplot 1 title', fontsize=22) # set subfigure title
```

- ⦿ for `add_subplot(...)` `subplot(...)` see
https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.subplot.html

see `plot_somelddata_v2.py`

```
ax2.legend(['cos', 'identity'], fontsize=20) # set legend for multiple graphs  
ax2.set_xlabel('xlabel', fontsize=9) #set x axis label  
ax2.set_ylabel('ylabel', fontsize=9) #set y axis label
```

see `plot_somedata_v2.py`

```
ax1.set_xlim([0.3,0.7]) # x axis ranges  
ax1.set_ylim([0.3,5])  # y axis ranges
```

see `plot_somelddata_v2.py`

```
ax2.plot(x, np.cos(x), linewidth=12, color=(0.8,0,0,1.0))  
ax2.plot(x, np.cos(x), marker='x', markersize=12)
```

seaborn macht schoenere plots, hat mehr high level Funktionen